Ejemplo sobre el uso de pila en Assembler

Arquitectura del Computador - LCC - FCEIA-UNR

2 de noviembre de 2026

Este ejemplo tiene como objetivo ilustrar algunos aspectos del uso de la pila, en particular, la reserva de espacio y el almacenamiento de variables locales dentro de una función. Asimismo, permite familiarizarse con el uso de GDB para examinar el contenido de la pila durante la ejecución del programa.

Dado el siguiente código en Assembler X86-64 en el archivo `stack_2.s`:

```
.data
a: .quad 45
b: .quad 56

.text
.global main

main:
    movq a, %rdi  # Primer argumento
    movq b, %rsi  # Segundo argumento
    call suma

    xorq %rax, %rax
    ret

suma:
    # Prólogo
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp

    # Cuerpo de la función
    movq $99, -8(%rbp)   # Variable local
    movq %rdi, %rax
    addq %rsi, %rax
    addq -8(%rbp), %rax  # a+b+99

    # Epílogo
    movq %rbp, %rsp
    popq %rbp
    ret
```

Compilar utilizando la opción `-g` para poder *debuggear* utilizando GDB:

```
$ gcc -g stack.s
```

Luego ejecutar utilizando GDB:

```
gdb ./a.out
```

Una vez que estamos dentro de la sesión de *debugging*, ponemos un *breakpoint* en `main` y ejecutamos el comando `run`:

```
(gdb) br main
Breakpoint 1 at 0x4004b6: file stack.s, line 9.
(gdb) r
Starting program: /home/dferoldi/2020/a.out

Breakpoint 1, main () at stack.s:9
movq a, %rdi
```

Luego vamos ejecutando instrucción a instrucción utilizando el comando `next` hasta llegar a la instrucción `call suma` donde se hace el llamado a la función `suma`:

```
movq a, %rdi
(gdb) n
movq b, %rsi
(gdb) n
call suma
```

Veamos el valor del registro `rsp` (*stack pointer*):

```
(gdb) i r rsp
rsp            0x7fffffffe1e8 0x7fffffffe1e8
```

Vemos que el registro `rsp` está "apuntando" a la dirección `0x7fffffffe1e8`. Ahora sigamos con la ejecución del código con el comando `step` para poder entrar a la función `suma`:

```
(gdb) s
suma () at stack.s:16
subqq $16, %rsp
```

Veamos ahora el nuevo valor del registro `rsp`:

```
(gdb) i r rsp
rsp            0x7fffffffe1e0 0x7fffffffe1e0
```

Vemos que el valor del registro disminuyó en 8 bytes. Veamos que hay almacenado en la dirección a la que apunta:

```
(gdb) x $rsp
0x7fffffffe1e0: 0x004004cb
```

De acuerdo a lo que hemos visto en teoría, el valor `0x004004cb` es la dirección de retorno de la función `suma`, es decir la dirección a la que hay retornar cuando finalice la función. ¿Cómo podemos verificarlo? Podemos ver si en la dirección `0x004004cb` efectivamente está la instrucción siguiente al llamado a la función, es decir la instrucción `xorq %rax, %rax`. Esto lo podemos lograr utilizando el comando `x` con la opción `i`:

```
(gdb) x/i 0x4004cb
   0x4004cb <main+21>: xor    %rax,%rax
```

Vemos que efectivamente en la dirección que se "pusheó" se encuentra almacenada la instrucción a la cual tiene que retornar el flujo del proceso una vez que finalice la función `suma`.

Ahora ejecutemos una instrucción más, verifiquemos el nuevo valor del registro `rsp` y lo que hay almacenado en la dirección a la que apunta:

```
(gdb) n
movq %rsp, %rbp
(gdb) i r rsp
rsp            0x7fffffffe1d8 0x7fffffffe1d8
(gdb) x/1xg $rsp
0x7fffffffe1d8: 0x0000000000000000
```

Vemos que el valor de `rsp` disminuyó otros 8 bytes y que en la dirección a la que apunta está almacenado el valor del registro `rbp` anterior al llamado a función, por lo cual ahora podremos trabajar con el registro `rbp` y modificarlo sin inconvenientes porque posteriormente podremos recuperarlo. Recordemos que el registro `rbp` es *calle saved*, por lo cual es responsabilidad de la función llamada de salvar y luego restaurar el valor que tenía antes de llamar a la función.

Avancemos una instrucción más y veamos el contenido de los registros `rsp` y `rbp`:

```
(gdb) i r rsp rbp
rsp            0x7fffffffe1d8 0x7fffffffe1d8
rbp            0x7fffffffe1d8 0x7fffffffe1d8
```

Vemos que ahora ambos registros apuntan a la misma dirección, la cual es el comienzo del marco de activación de la función `suma`.

Si ejecutamos una instrucción más, el valor de `rsp` disminuye en 16, reservando así 16 bytes en la pila para las variables locales de la función. A continuación, mediante la instrucción `movq $99, -8(%rbp)`, almacenamos el valor 99 en una de las posiciones reservadas, que utilizaremos como variable local de la función `suma`.

Podemos acceder a esta variable mediante un desplazamiento relativo al registro `rbp`, ya que este permanece fijo durante la ejecución de la función y sirve como referencia para acceder a los datos almacenados en su marco de pila. Por ejemplo, podemos examinar su contenido utilizando GDB:

```
(gdb) x/gd $rbp-8
0x7fffffffe1d0:  99
```

La siguiente instrucción suma los argumentos de la función que por convención de llamada vienen en los registros `rdi` y `rsi`:

```
addq %rdi, %rsi
```

Luego, al resultado en `rsi` se le suma el valor de la variable local referenciando de manera relativa al registro `rbp`:

```
addq -8(%rbp), %rsi
```

Posteriormente, en la instrucción `movq %rsi, %rax` se carga el resultado en el registro `rax`, debido a la convención de llamada.

Veamos ahora que hacen las siguientes dos instrucciones. Este el denominado "epílogo". Primero, se "mueve" el puntero `rsp` hacia el comienzo del marco de activación, el cual está apuntado por `rbp`:

```
movq %rbp, %rsp
(gdb) i r rbp rsp
rbp            0x7fffffffe1d8 0x7fffffffe1d8
rsp            0x7fffffffe1d8 0x7fffffffe1d8
```

Vemos que ahora ambos registros apuntan nuevamente al comienzo del marco de activación. Luego, se restaura en el registro `rbp` el valor que tenía antes de llamar a la función y que habíamos "salvado" en la pila:

```
popq %rbp
(gdb) i r rbp
rbp            0x0 0x0
```

En este punto el registro `rsp` está apuntando a la dirección donde se "pusheo" la direción de retorno de la función `suma`. Por lo tanto, al ejecutar la instrucción `ret` se retornará a la instrucción siguiente a `call suma`. Si chequeamos el contenido del registro `rax`, vemos que efectivamente tenemos el resultado de las operaciones que se realizaron en la función `suma`:

```
(gdb) i r rax
rax            0xc8 200
```

Es importante notar que si no hubiéramos ejecutado las instrucciones del epílogo, el registro `rsp` no hubiera estado apuntando a la dirección de retorno al pretender retornar con la instrucción `ret` y por lo tanto se hubiera producido una violación de segmento.

Finalmente, luego de ejecutar la instrucción `xorq %rax, %rax` se obtuvo un valor 0 en el registro `rax`, debido a las propiedades de la operación *exclusive or*, el cual será el valor de retorno de `main` una vez ejecutada la instrucción `ret`. Esto se puede verificar con el comando `echo` inmediatamente después de haber sido ejecutado el proceso:

```
$ ./a.out
$ echo $?
0
```

En la siguiente figura podemos ver el esquema de la pila con las direcciones de memoria a la izquierda y a la derecha cómo van apuntando los registros `rbp` y `rsp` a lo largo de la ejecución del proceso. Los valores de las direcciones de memoria pueden diferir según la sesión de depuración.

